Machine Learning

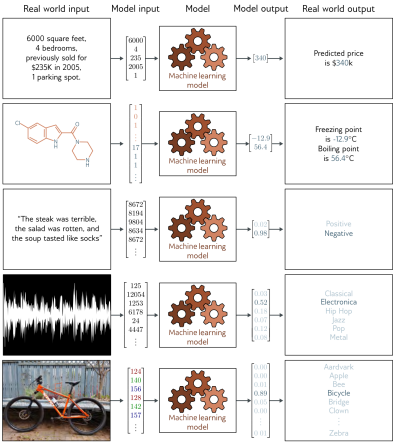
2. Classification, with and beyond the linear model

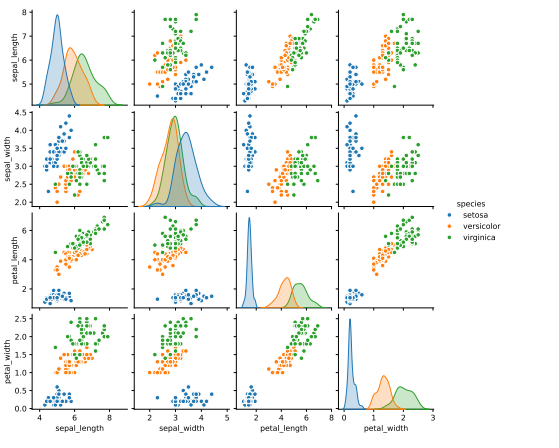
Louis Bagot
Université Claude Bernard Lyon 1

September 28, 2026

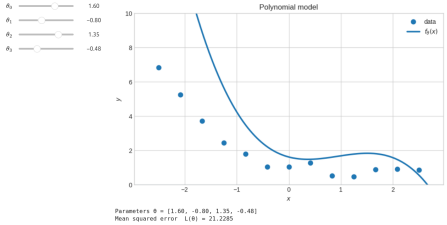
Where were we?

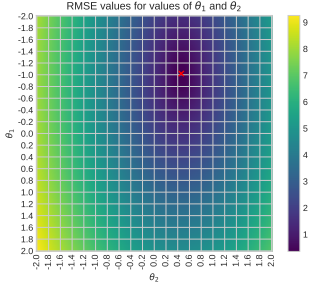
ML: Approximate a function from data examples $\{x_i, y_i\}$

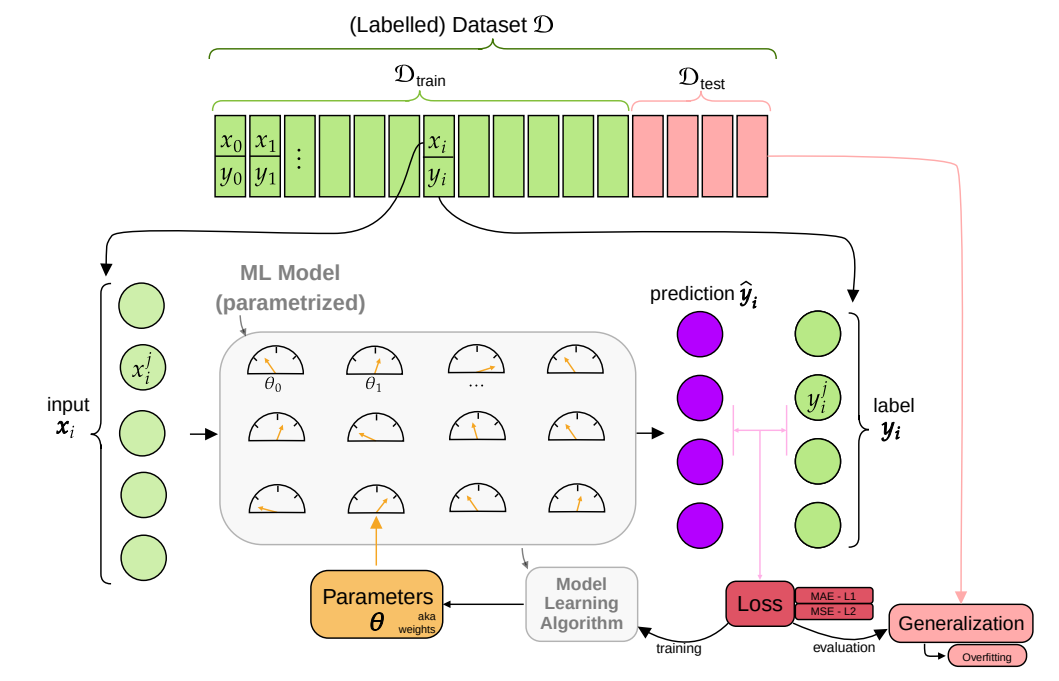




Parametrized models $f_\theta(x_i) \approx y_i$ learn parameters θ to minimize loss: $\arg\min_\theta \mathcal{L}(\mathcal{D}, f_\theta)$






(Labelled) Dataset $\mathcal{D}$
$\mathcal{D}_{\text{train}}$
$\mathcal{D}_{\text{test}}$
x_0 y_0
x_1 y_1
x_i y_i
ML Model
(parametrized)
θ_0
θ_1
...
input
$\boldsymbol{x}_i$
x_i^j
prediction $\hat{\boldsymbol{y}}_i$
y_i^j
label
$\boldsymbol{y}_i$
Parameters
$\boldsymbol{\theta}$
aka
weights
Model
Learning
Algorithm
Loss
MAE - L1
MSE - L2
training
evaluation
Generalization
Overfitting

Notes for this slide

Parameters $\boldsymbol{\theta}$ can be conceptualized like the knobs of an oven: you set them to some value and let the model cook with those.
The performance of the model (“quality of the cooking”) is determined by the loss.
The learning algorithm tells you how to change the knobs to get better performance.

Where were we?

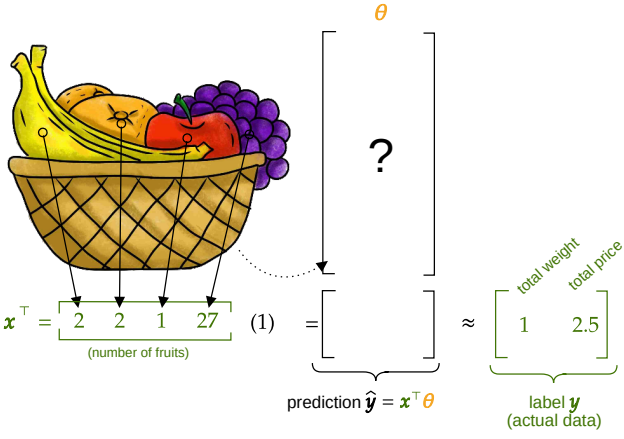
We studied the **Linear Model**:

$$f_{\boldsymbol{\theta}}(\boldsymbol{x}) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots = \boldsymbol{x}^\top \boldsymbol{\theta}$$

Where were we?

We studied the **Linear Model**:

$$f_{\boldsymbol{\theta}}(\boldsymbol{x}) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots = \boldsymbol{x}^\top \boldsymbol{\theta}$$



Notes for this slide

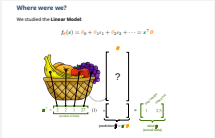


Image our data contains

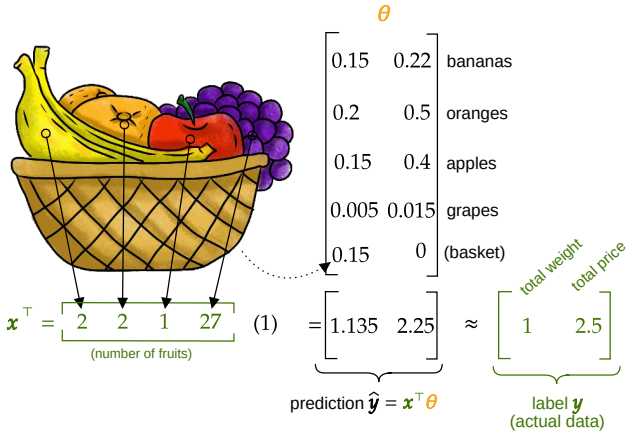
$$\{\boldsymbol{x_i} = [\text{number of different fruits}], \boldsymbol{y_i} = [\text{weight and price of the basket (total)}]\}$$

So we need to estimate what the weight and price of a basket will be from the number of fruits, and we have a bunch of example baskets drawn from people doing groceries. Intuitively, the relationships are linear: the weights and prices of each fruit add up.

Where were we?

We studied the **Linear Model**:

$$f_{\boldsymbol{\theta}}(\boldsymbol{x}) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots = \boldsymbol{x}^\top \boldsymbol{\theta}$$



Notes for this slide

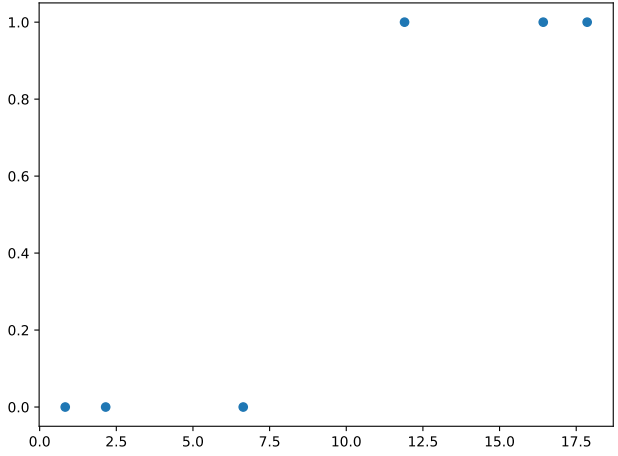


The linear model can disentangle the individual prices of fruits to guess the average weight and price of each single fruit, giving us a good estimate of the price.

To illustrate the importance of θ_0, we can assume that everything is weighted along with the basket itself: there is always a flat weight of 0.15kg even if the basket is empty (all quantities at 0).

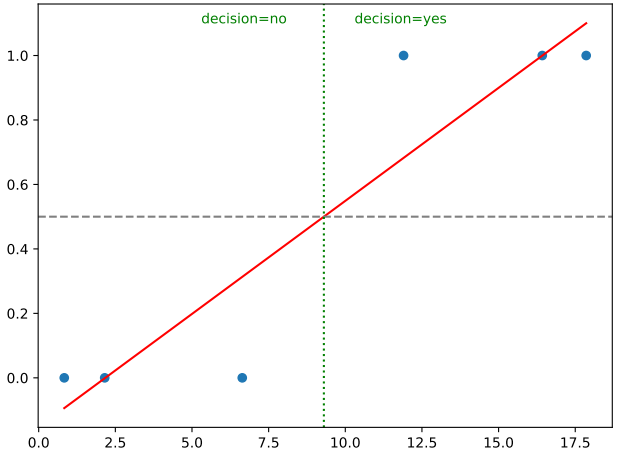
What about classification?

If I need to answer “is this basket more expensive than 10€?” (no:0, yes:1)



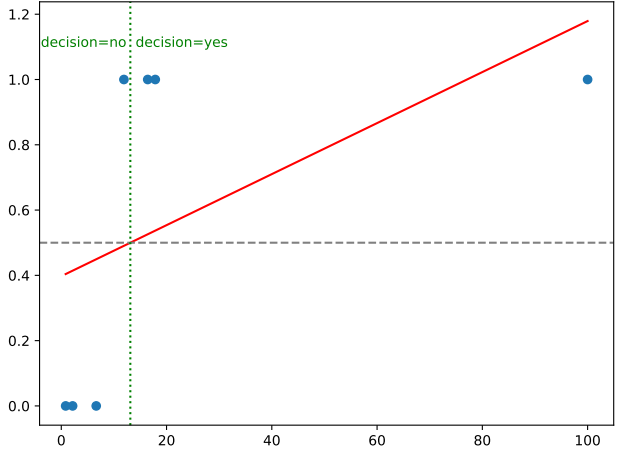
What about classification?

If I need to answer “is this basket more expensive than 10€?” (no:0, yes:1)
Naive algo: fit the line and decide “yes” when line > 0.5:



Linear classif.: what if there are outliers?

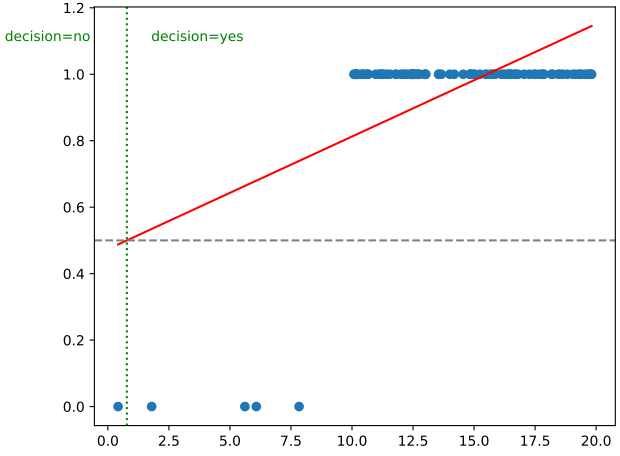
Linear classif.: what if there are outliers?



(Note: MSE & OLS are weak to outliers even outside of classification!)

Linear classif.: what if there are imbalances?

Linear classif.: what if there are imbalances?

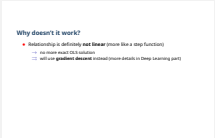


Why doesn’t it work?

Why doesn't it work?

- Relationship is definitely **not linear** (more like a step function)
  - $\rightarrow$ no more exact OLS solution
  - $\rightrightarrows$ will use **gradient descent** instead (more details in Deep Learning part)

Notes for this slide



We will see gradient descent in more detail in the Deep Learning part.
For now what you need to know: for your loss $\mathcal{L}$ and a single parameter θ_i of your model f_θ, it tells you **what little change of θ_i you can make to slightly decrease $\mathcal{L}$**.
For this it uses the "gradient" in math, the derivative vector w.r.t each parameter:

$$\nabla_\theta \mathcal{L} = \left[\frac{\partial \mathcal{L}}{\partial \theta_i}\right]_i$$

.

Why doesn't it work?

- Relationship is definitely **not linear** (more like a step function)
  - $\rightarrow$ no more exact OLS solution
  - $\rightrightarrows$ will use **gradient descent** instead (more details in Deep Learning part)

- MSE doesn't make sense for classification
  - $\rightarrow$ an error of 1 should be considered *catastrophic* (worst possible)
  - $\rightrightarrows$ **use a suitable loss** ...?

Why doesn't it work?

- Relationship is definitely **not linear** (more like a step function)
  - $\rightarrow$ no more exact OLS solution
  - $\rightrightarrows$ will use **gradient descent** instead (more details in Deep Learning part)

- MSE doesn't make sense for classification
  - $\rightarrow$ an error of 1 should be considered *catastrophic* (worst possible)
  - $\rightrightarrows$ **use a suitable loss** ...?

- Linear model outputs go way past $[0, 1]$

Outputs in $[0, 1]$: just turn them into probabilities?

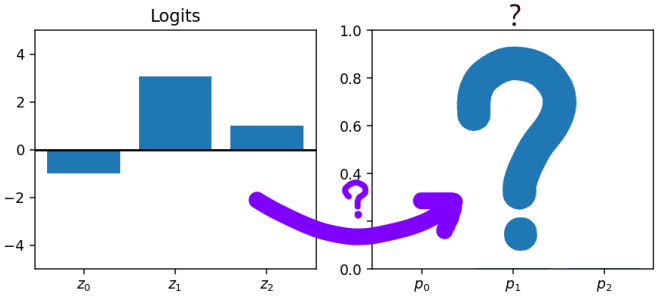
Outputs in $[0, 1]$: just turn them into probabilities?

Supposed the linear model outputs $x^\top \theta_c = z_c$ for each class c (matrix θ).
We would like to interpret those **as probabilities** $\in [0, 1]$ and $\sum_c z_c = 1$.

Outputs in $[0, 1]$: just turn them into probabilities?

Supposed the linear model outputs $x^\top \boldsymbol{\theta}_c = z_c$ for each class c (matrix $\boldsymbol{\theta}$).
We would like to interpret those **as probabilities** $\in [0, 1]$ and $\sum_c z_c = 1$.

logit z_c $\xrightarrow{\text{transformation?}}$ $\hat{y}_c = \text{Proba(class c)}$



we want to turn the logits into a **probability distribution** over classes.

Make them positive and divide by the sum?

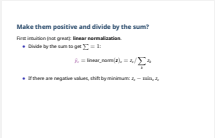
Make them positive and divide by the sum?

First intuition (not great): **linear normalization**.

- Divide by the sum to get $\sum = 1$:

$$\hat{y}_c = \text{linear_norm}(\boldsymbol{z})_c = z_c / \sum_k z_k$$

- If there are negative values, shift by minimum: $z_c - \min_c z_c$



Notes for this slide

You do not need to remember the linear normalization, it's only to explain how we get to the softmax! 😁

Make them positive and divide by the sum?

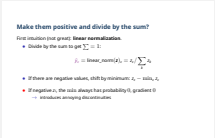
First intuition (not great): **linear normalization**.

- Divide by the sum to get $\sum = 1$:

$$\hat{y}_c = \text{linear_norm}(\boldsymbol{z})_c = z_c / \sum_k z_k$$

- If there are negative values, shift by minimum: $z_c - \min_c z_c$
- If negative z_i, the $\min$ always has probability 0, gradient 0
  $\rightarrow$ introduces annoying discontinuities

Notes for this slide



A probability strictly $= 0$ can be an issue in many cases.

- If we sustracted the minimum from itself, then gradient= 0: we cannot know how a little change of z_c would improve the probability score
- Reasoning in terms of log probabilities (as we will do in the next slides) introduces a negative infinite
- In many domains, we still want to try the option out with a little bit of probability to ensure we're not missing out on something important
  - a word in language
  - an action (e.g. a chess move, a robot motor) in reinforcement learning

Make them positive and divide by the sum?

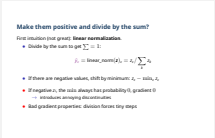
First intuition (not great): **linear normalization**.

- Divide by the sum to get $\sum = 1$:

$$\hat{y}_c = \text{linear_norm}(\boldsymbol{z})_c = z_c / \sum_k z_k$$

- If there are negative values, shift by minimum: $z_c - \min_c z_c$
- If negative z_i, the $\min$ always has probability 0, gradient 0
  - $\rightarrow$ introduces annoying discontinuities
- Bad gradient properties: division forces tiny steps

Notes for this slide



Note that the linear normalization is invariant to scaling:

$$\text{linear_norm}(\boldsymbol{z}) = \text{linear_norm}(\alpha \boldsymbol{z})$$

However the bigger the logits are, the bigger the division is, so the model may wander into places where it can't learn properly anymore.

Classif.: how do we improve from linear normalization?

Classif.: how do we improve from linear normalization?

Linear normalization with **exponentials**, called the **softmax** function:

$$\hat{y}_c = \text{Proba(class c)} = \text{softmax}(\boldsymbol{z})_c \doteq \frac{e^{z_c}}{\sum_k e^{z_k}}$$



Notes for this slide

Note that now softmax is invariant to *shift*:

$$\text{softmax}(\boldsymbol{z}) = \text{softmax}(\alpha + \boldsymbol{z}).$$

This means that it doesn't care about scale, only about relative logit differences.

Softmax is sometimes "augmented" with a **temperature parameter** τ:

$$\text{softmax}(\boldsymbol{z})_c \doteq \frac{e^{\tau z_c}}{\sum_k e^{\tau z_k}}$$

At $\tau = 0$, the distribution is uniform; at $\tau = \infty$, it is an $\arg\max$!
This is where its name comes from: a soft version of the maximum operator.

Classif.: how do we improve from linear normalization?

Linear normalization with **exponentials**, called the **softmax** function:

$$\hat{y}_c = \text{Proba(class c)} = \text{softmax}(\boldsymbol{z})_c \doteq \frac{e^{z_c}}{\sum_k e^{z_k}}$$

- Naturally strictly positive (no shifting-by-min or 0 proba issues)
  - no shift-by-min issues
  - no proba $= 0$ issues
  - $\rightarrow$ smooth changes across all z_c choices

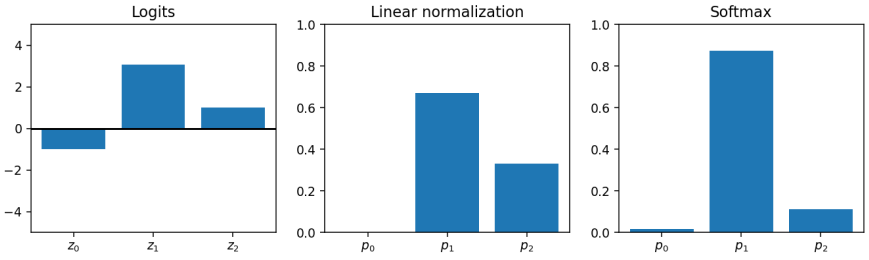
Classif.: how do we improve from linear normalization?

Linear normalization with **exponentials**, called the **softmax** function:

$$\hat{y}_c = \text{Proba(class c)} = \text{softmax}(\boldsymbol{z})_c \doteq \frac{e^{z_c}}{\sum_k e^{z_k}}$$

- Naturally strictly positive (no shifting-by-min or 0 proba issues)
  - no shift-by-min issues
  - no proba $= 0$ issues
  - $\rightarrow$ smooth changes across all z_c choices
- Still a division though?

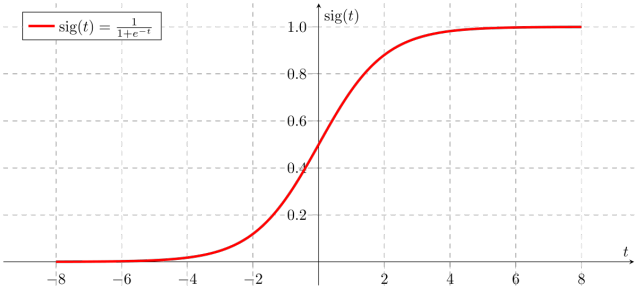
What does softmax look like?



(you can play with it in the associated interactive notebook)

What does softmax look like?

For two classes (binary yes/no), it collapses to the **sigmoid function**:

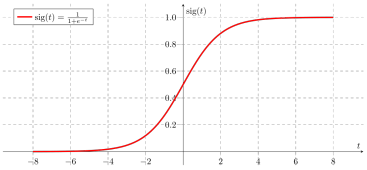


$$\text{sigmoid}(z) \doteq \frac{1}{1+e^{-z}}$$

$$\lim \text{sigmoid}(z) \begin{array}{l} = 0 \text{ as } z \to -\infty \\ = 0.5 \text{ as } z = 0 \\ = 1 \text{ as } z \to +\infty \end{array}$$

What does softmax look like?

For two classes (binary yes/no), it collapses to the **sigmoid function**:

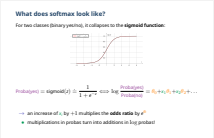


$$\text{Proba(yes)} = \text{sigmoid}(z) \doteq \frac{1}{1+e^{-z}} \iff \log \frac{\text{Proba(yes)}}{\text{Proba(no)}} = \theta_0 + x_1\theta_1 + x_2\theta_2 + \dots$$

$\rightarrow$ an increase of x_i by $+1$ multiplies the **odds ratio** by e^{θ_i}

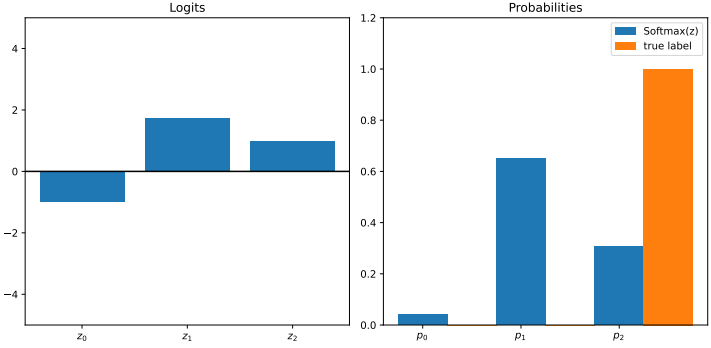
- multiplications in probas turn into additions in $\log$ probas!

Notes for this slide



You don't need to know that proba/proba property by heart, only understand the point that multiplications of probas (which happen often: probas independent events multiply) turn into additions, exactly what the linear model is doing (addition of weighted x_i).

What loss to use with softmax?



We need a good notion of distance between the softmax and the true label.

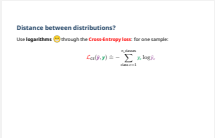
Distance between distributions?

Distance between distributions?

Use **logarithms** 😁 through the **Cross-Entropy loss**: for one sample:

$$\mathcal{L}_{\text{CE}}(\hat{\boldsymbol{y}}, \boldsymbol{y}) \doteq -\sum_{\text{class } c=1}^{\text{n_classes}} y_c \log \hat{y}_c$$

Notes for this slide



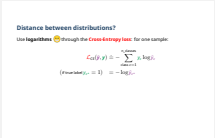
For our matters the Cross-Entropy Loss (CE) is essentially identical to the Negative Log Likelihood Loss (NLL), which you may encounter. Be careful though: **there is a difference in implementation in PyTorch**, where Cross-Entropy will apply the softmax for you while NLL expects you to have passed your inputs through a LogSoftmax.

Distance between distributions?

Use **logarithms** 😁 through the **Cross-Entropy loss**: for one sample:

$$\mathcal{L}_{\mathsf{CE}}(\hat{\boldsymbol{y}}, \boldsymbol{y}) \doteq -\sum_{\text{class } c=1}^{\text{n_classes}} y_c \log \hat{y}_c$$

$$\left(\text{if true label } y_{c^\star} = 1\right) \quad = -\log \hat{y}_{c^\star}$$



Notes for this slide

By "true label" I mean the one that the one that was correct (had probability 1 and others 0). Be mindful of the negative sign: this is a loss, we want to minimize it, so after the negative sign we want to maximize this (log)probability! Since the log is a strictly increasing function, this means increasing $\hat{y}_{c^*}$.

Distance between distributions?

Use **logarithms** 😁 through the **Cross-Entropy loss**: for one sample:

$$\mathcal{L}_{\text{CE}}(\hat{\boldsymbol{y}}, \boldsymbol{y}) \doteq -\sum_{\text{class } c=1}^{\text{n_classes}} y_c \log \hat{y}_c$$

$$\left(\text{if true label } y_{c^\star} = 1\right) \quad = -\log \hat{y}_{c^\star}$$

- Simply aims to **rise the probability of the correct class**.

Distance between distributions?

Use **logarithms** 😁 through the **Cross-Entropy loss**: for one sample:

$$\mathcal{L}_{\text{CE}}(\hat{\boldsymbol{y}}, \boldsymbol{y}) \doteq -\sum_{\text{class } c=1}^{\text{n_classes}} y_c \log \hat{y}_c$$

$$\left(\text{if true label } y_{c^\star} = 1\right) \quad = -\log \hat{y}_{c^\star}$$

- Simply aims to **rise the probability of the correct class**.
- Cross-Entropy acts as a **distance between distributions** (here, $\hat{\boldsymbol{y}} \longleftrightarrow \boldsymbol{y}$)

Distance between distributions?

Use **logarithms** 😁 through the **Cross-Entropy loss**: for one sample:

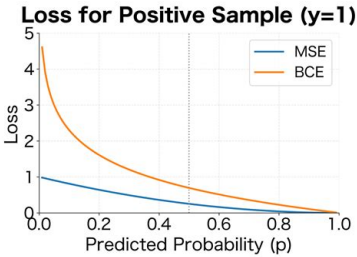
$$\mathcal{L}_{\text{CE}}(\hat{\boldsymbol{y}}, \boldsymbol{y}) \doteq -\sum_{\text{class } c=1}^{\text{n_classes}} y_c \log \hat{y}_c$$

$$\left(\text{if true label } y_{c^\star} = 1\right) \quad = -\log \hat{y}_{c^\star}$$

- Simply aims to **rise the probability of the correct class**.
- Cross-Entropy acts as a **distance between distributions** (here, $\hat{\boldsymbol{y}} \longleftrightarrow \boldsymbol{y}$)
- The $\log$ cancels out the softmax $\exp$ and turns the division into a subtraction!

What does Cross-Entropy do?

$$\mathcal{L}_{\mathrm{CE}}(\hat{\boldsymbol{y}}, \boldsymbol{y}) = -\log \hat{y}_{c^\star}$$



Notes for this slide

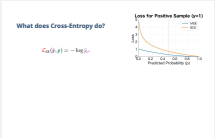
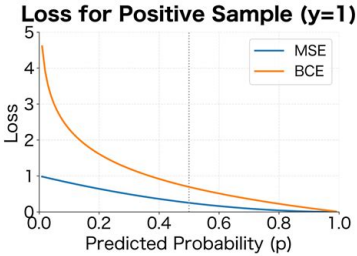


Image from the M. Brenndoerfer page on loss functions. The image mentions ”BCE” for *Binary* Cross-Entropy as the name of the loss with only two classes.

What does Cross-Entropy do?

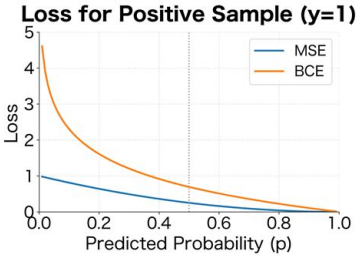
$$\mathcal{L}_{\text{CE}}(\hat{\boldsymbol{y}}, \boldsymbol{y}) = -\log \hat{y}_{c^\star}$$



- As a distance, punishes mistakes heavily: $\hat{y}_{c^\star} \to 0 \implies \mathcal{L}_{\text{CE}} \to \infty$

What does Cross-Entropy do?

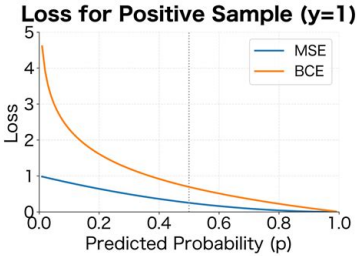
$$\mathcal{L}_{\text{CE}}(\hat{\boldsymbol{y}}, \boldsymbol{y}) = -\log \hat{y}_{c^\star}$$



- As a distance, punishes mistakes heavily: $\hat{y}_{c^\star} \to 0 \implies \mathcal{L}_{\text{CE}} \to \infty$
- Treats the logits as *log-probabilities*
  - $\to$ this is why the z_i are called $\log$its! $\log$ probabilities!

What does Cross-Entropy do?

$$\mathcal{L}_{\text{CE}}(\hat{\boldsymbol{y}}, \boldsymbol{y}) = -\log \hat{y}_{c^\star}$$



- As a distance, punishes mistakes heavily: $\hat{y}_{c^\star} \to 0 \implies \mathcal{L}_{\text{CE}} \to \infty$
- Treats the logits as *log-probabilities*
  - $\to$ this is why the z_i are called $\log$its! $\log$ probabilities!
- As a differentiable function: $\frac{\partial \mathcal{L}_{\text{CE}}}{\partial z_c} = \hat{y}_c - y_c$, just like MSE!
  - if prediction was $\hat{y}_c = 1$ for $y_c = 0$, push logit down (descent), otherwise up
  - recovers nice gradients in logit / log probability space.

Summary please?

We need to adapt the output and loss of our model to what we're doing.

Summary please?

We need to adapt the output and loss of our model to what we're doing.

- **Regression**:
  - In general, do not do anything to the output.
    Leave $y_i = z_i$ (e.g. $\boldsymbol{x}^\top \boldsymbol{\theta}_i$ for a linear model)
  - Use MSE or MAE losses

Summary please?

We need to adapt the output and loss of our model to what we're doing.

- **Regression**:
  - In general, do not do anything to the output.
    Leave $y_i = z_i$ (e.g. $\boldsymbol{x}^\top \boldsymbol{\theta}_i$ for a linear model)
  - Use MSE or MAE losses

- **Classification**:
  - Turn the outputs into probabilities using softmax: $y_c = \text{softmax}(\boldsymbol{z})_c$.
    If there are only two classes, you can use the sigmoid with 0=no, 1=yes.
  - Use the Cross-Entropy loss: this is called **logistic regression!**

Summary please?

We need to adapt the output and loss of our model to what we're doing.

- **Regression**:
  - In general, do not do anything to the output.
    Leave $y_i = z_i$ (e.g. $\boldsymbol{x}^\top \boldsymbol{\theta}_i$ for a linear model)
  - Use MSE or MAE losses

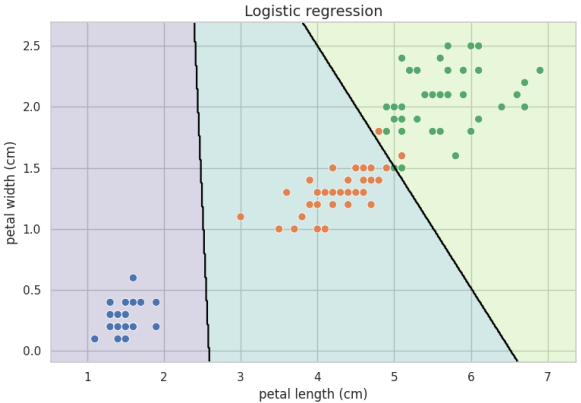
- **Classification**:
  - Turn the outputs into probabilities using softmax: $y_c = \text{softmax}(\boldsymbol{z})_c$.
    If there are only two classes, you can use the sigmoid with 0=no, 1=yes.
  - Use the Cross-Entropy loss: this is called **logistic regression!**

$\rightarrow$ All this is true **all the way up to Deep Learning!**:

Summary please?

Classification with Linear models:

$$f_{\theta}(\boldsymbol{x}) = \text{softmax}(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots)$$



How do we evaluate a model on classification?

Cross Entropy is not easy to read...

How do we evaluate a model on classification?

Cross Entropy is not easy to read...

Starting with **binary classification** (yes/no: detect spam, sickness, license, ...)

How do we evaluate a model on classification?

Cross Entropy is not easy to read...
Starting with **binary classification** (yes/no: detect spam, sickness, license, ...)

Most straightforward: **accuracy**

$$\text{Accuracy} = \frac{\text{correct classifications}}{\text{total classifications}}$$

Notes for this slide



Example from Remy Cazabet's course:

- Test 1000 people for Covid
- ML model: TP: 45 - FN: 5 - TN: 900 -FP: 50; Accuracy= $(45 + 900)/1000 = 0.945$
- Naive classifier: Always answer: not covid; Accuracy= $(0 + 950)/1000 = 0.95$

How do we evaluate a model on classification?

Cross Entropy is not easy to read...
Starting with **binary classification** (yes/no: detect spam, sickness, license, ...)

Most straightforward: **accuracy**

$$\text{Accuracy} = \frac{\text{correct classifications}}{\text{total classifications}}$$

- Has a huge issue! Can you see it?

How do we evaluate a model on classification?

Cross Entropy is not easy to read...
Starting with **binary classification** (yes/no: detect spam, sickness, license, ...)

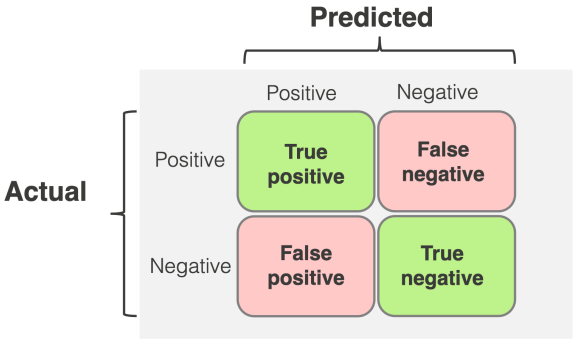
Most straightforward: **accuracy**

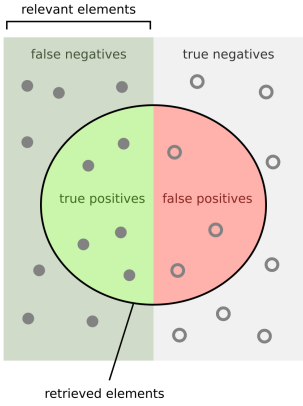
$$\text{Accuracy} = \frac{\text{correct classifications}}{\text{total classifications}}$$

- Has a huge issue! Can you see it?
  - $\rightarrow$ **Class imbalance**: predict the most frequent class(es), ignore the rest
- (Accuracy is still very useful in many other cases, just be aware!)

Can we improve around the accuracy?

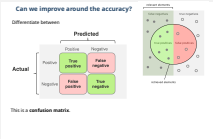
Differentiate between





This is a **confusion matrix**.

Notes for this slide



Left image from EvidentlyAI.
Right image from Wikipedia.

If the true/false/pos/neg is confusing, a litte help: the model predicts either positive or negative, the actual data says whether it's true or false. If there's an F(alse), the model made a mistake!

Can we improve around the accuracy?

Recall: (probability of detection)

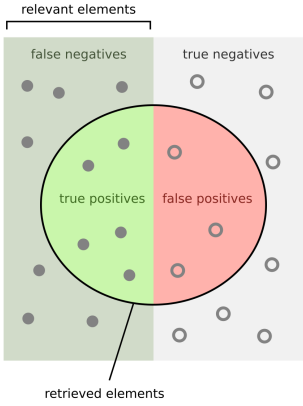
Among the "true", how much did we find?

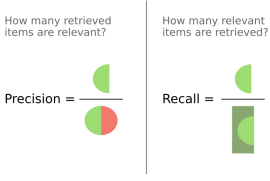
$$\text{Recall} = \frac{TP}{TP + FN}$$

Precision:

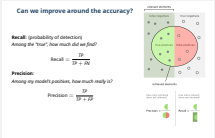
Among my model's positives, how much really is?

$$\text{Precision} = \frac{TP}{TP + FP}$$





Notes for this slide



You can play with these concepts a bit more on the Google ML course.

If you're interested in ranking-based scores, see Remy Cazabet's course.

Can we improve around the accuracy?

Recall: (probability of detection)

Among the "true", how much did we find?

$$\text{Recall} = \frac{TP}{TP + FN}$$

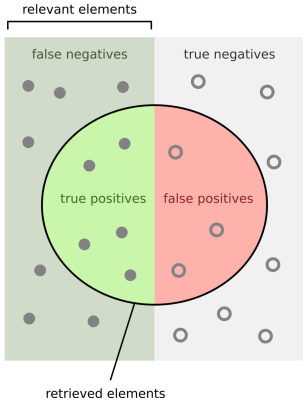
Precision:

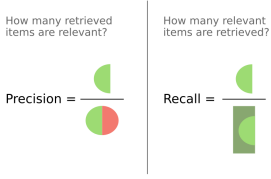
Among my model's positives, how much really is?

$$\text{Precision} = \frac{TP}{TP + FP}$$

F1 Score:

$$F_1 = 2\frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

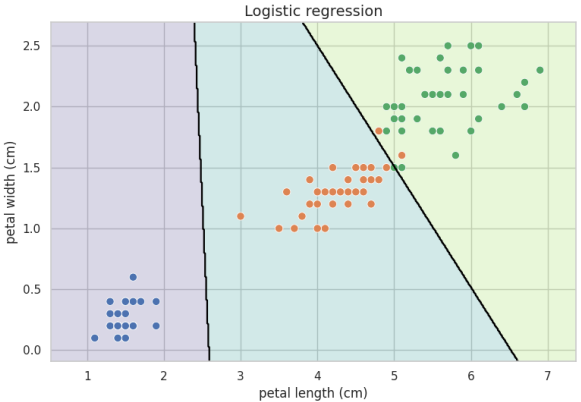




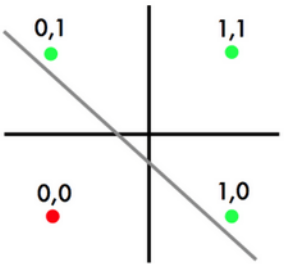
Back to building models!
Can we improve on the linear models?

Linear models:

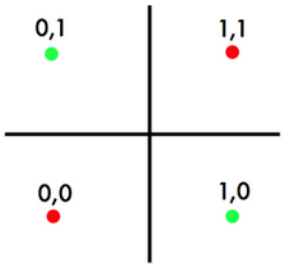
$$f_{\boldsymbol{\theta}}(\boldsymbol{x}) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots$$



What about non-linear relationships?



OR



XOR

Notes for this slide

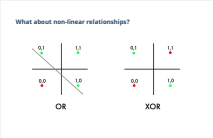


Image from this website, representing the ”xor problem”: if we want to split the space into the green and red classes, a single linear split does not work.

What about non-linear relationships?

Many, many families of models (virtually all parametric, $f_{\theta}(\boldsymbol{x})$) with:

- Bunch of parameters θ
- Sequence of computations $f_{\theta}(\boldsymbol{x})$
  - $\rightarrow$ this controls the *shape of the model prediction*
  - $\rightrightarrows$ pick the right model for the task!

What about non-linear relationships?

Many, many families of models (virtually all parametric, $f_\theta(\boldsymbol{x})$) with:

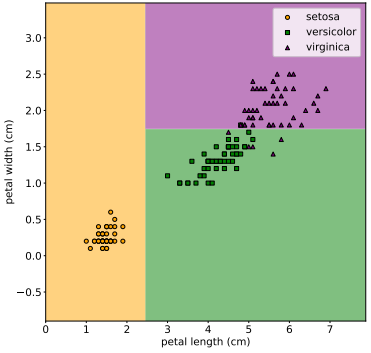
- Bunch of parameters θ
- Sequence of computations $f_\theta(\boldsymbol{x})$
  - $\rightarrow$ this controls the *shape of the model prediction*
  - $\rightrightarrows$ pick the right model for the task!

- Algorithm for learning the parameters (minimizing the loss)

Ideas of non-linear models?

Ideas of non-linear models?

Intuitive idea: **keep splitting the inputs along feature values**



Notes for this slide

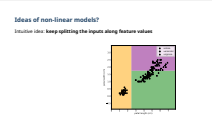
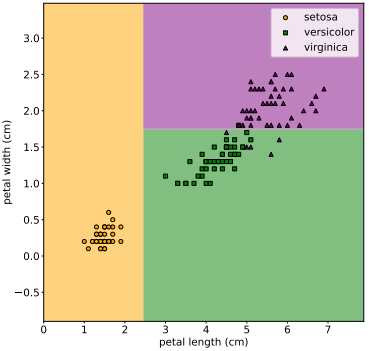


Figure produced with the notebook from the Probabilistic ML book; see the associated notebook in the github course folder.

Ideas of non-linear models?

Intuitive idea: **keep splitting the inputs along feature values**

This is a **Decision Tree**!



Training Decision Trees: how do we make a split?

Training Decision Trees: how do we make a split?

- What would a **“good split”** mean?

Training Decision Trees: how do we make a split?

- What would a **“good split”** mean?
  - Classif: not many overlapping classes in splits
  - Regression: good average predictions in splits

Training Decision Trees: how do we make a split?

- What would a **“good split”** mean?
  - Classif: not many overlapping classes in splits
  - Regression: good average predictions in splits
- **Where** do we split?

Training Decision Trees: how do we make a split?

- What would a **"good split"** mean?
  - Classif: not many overlapping classes in splits
  - Regression: good average predictions in splits

- **Where** do we split?
  - Consider all features d (data dimensions)
  - Possible split between each sample value (mean of both): $N - 1$ tries

  $\rightarrow$ total $d \times (N - 1)$ tests

How do we decide which split to make?

How do we decide which split to make?

Example with classif:
For split class proportions $\boldsymbol{p} = [p_1, \ldots, p_{\text{n_classes}}]$, with $p_i = $ % of class i in split, compute **Gini impurity:**

$$I_G(\boldsymbol{p}) = 1 - \sum_c p_c^2$$

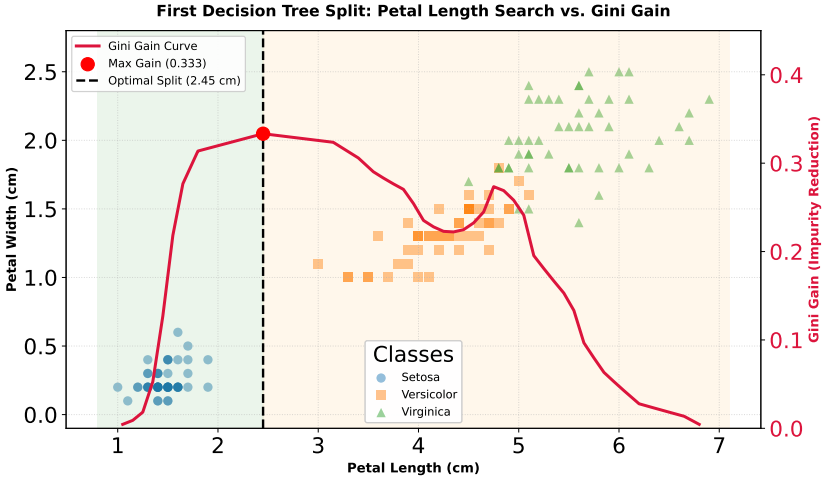
How do we decide which split to make?

Example with classif:
For split class proportions $\boldsymbol{p} = [p_1, \ldots, p_{\text{n_classes}}]$, with p_i = % of class i in split, compute **Gini impurity:**

$$I_G(\boldsymbol{p}) = 1 - \sum_c p_c^2$$

- Maximal when classes are balanced ("*impure*" split)
- Minimal (=0) at *pure* (single-class) splits.

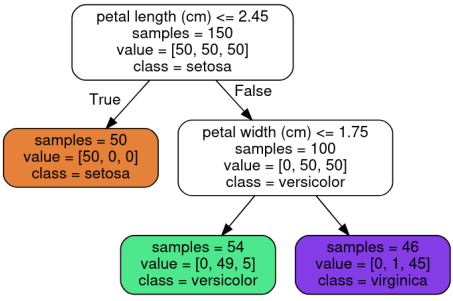
How do we decide which split to make?

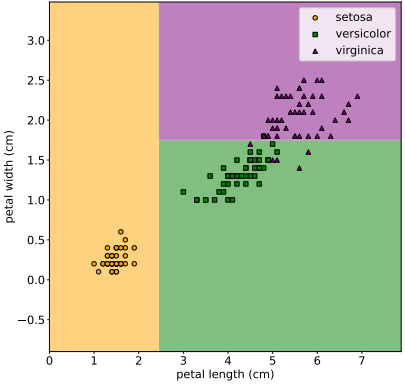


Next split: which part of the space do we split?

Next split: which part of the space do we split?

Try both! This is how we get a *tree structure*:





What is the final algo?

Training: repeat until target depth / node budget:

- At **each node and feature**, evaluate possible splits
  - — classification: Gini
  - — regression: MSE
  - → weigh each by the number of samples in each side of the split

What is the final algo?

Training: repeat until target depth / node budget:

- At **each node and feature**, evaluate possible splits
  - — classification: Gini
  - — regression: MSE
  - → weigh each by the number of samples in each side of the split

- Split by the best metric increase, expanding the tree with new node/rule

What is the final algo?

Training: repeat until target depth / node budget:

- At **each node and feature**, evaluate possible splits
  - — classification: Gini
  - — regression: MSE
  - → weigh each by the number of samples in each side of the split
- Split by the best metric increase, expanding the tree with new node/rule

Inference: go down the tree for the new input

- classification: most popular class in leaf node
- regression: average value in leaf node

How good are Decision Trees?

How good are Decision Trees?

- Can easily model some non-linear relationships
- Work with little data, and can quickly extract important relationships
- Easily understood and quite explainable: like “expert” systems
  - $\rightarrow$ not true anymore as soon as the tree gets big
- Fast inference



Notes for this slide

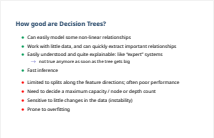
Expert systems pre-date ML as the main implementation of “AI”: writing, along with an expert of the domain, a huge set of rules to decide the best output. For example, you might write with a medical team how to diagnose certain illnesses; it works as a list of conditions and multiple-choice questions to reach a pre-established outcome.

How good are Decision Trees?

- Can easily model some non-linear relationships
- Work with little data, and can quickly extract important relationships
- Easily understood and quite explainable: like “expert” systems
  - $\rightarrow$ not true anymore as soon as the tree gets big
- Fast inference

- Limited to splits along the feature directions; often poor performance
- Need to decide a maximum capacity / node or depth count
- Sensitive to little changes in the data (instability)
- Prone to overfitting

Notes for this slide



Note on instability: in the Probabilistic ML book, page 607 (Section 18.1.5) the author shows that a decision tree can find different splits by removing just one datapoint.

Instability & Overfit: could we *average* over trees?

Instability & Overfit: could we *average* over trees?

Expose different trees to different data: **Random Forest**

- Prepare your forest:
  - Pick the tree hyper-parameters (depth, #nodes)
  - Pick a number of trees and an averaging function

Instability & Overfit: could we *average* over trees?

Expose different trees to different data: **Random Forest**

- Prepare your forest:
  - Pick the tree hyper-parameters (depth, #nodes)
  - Pick a number of trees and an averaging function
- Subsample datapoints or features $\longrightarrow$ importance of diversity!
- Train all trees.
- For inference decision, use the averaging function

Instability & Overfit: could we *average* over trees?

Expose different trees to different data: **Random Forest**

- Prepare your forest:
  - Pick the tree hyper-parameters (depth, #nodes)
  - Pick a number of trees and an averaging function
- Subsample datapoints or features $\longrightarrow$ importance of diversity!
- Train all trees.
- For inference decision, use the averaging function

The idea is that **errors cancel out** at scale

- little probability that they all make the same mistakes!

$\longrightarrow$ "wisdom of the crowds"

- This is a first example of **Ensemble Learning!** more info next time :P